Git & GitHub — Complete Command Guide

From zero to production. A practical guide for teams and groups to learn and use Git/GitHub effectively.

Table of Contents

---

1. What is Git & GitHub?

Term	Meaning
Git	A version control system. Tracks changes in your code over time. Lives on your computer.
GitHub	A website that hosts Git repositories in the cloud. Lets team members share code.
Repository (repo)	A folder that Git tracks. Contains your project files and their history.
Commit	A snapshot of your code at a point in time. Like a "save point."
Branch	A parallel line of work. Lets you work on features without affecting the main code.
Remote	A copy of the repo on GitHub (or another server). `origin` usually means GitHub.
HEAD	Points to your current commit or branch. `HEAD~1` = one commit before HEAD.
Working directory	Your project folder where you edit files.
Staging area (index)	Place where files go after `git add` and before `git commit`.

---

2. Setup (Getting Started)

2.1 Install Git

Ubuntu/Debian:

```
sudo apt update
sudo apt install git
```

Check installation:

```
git --version
```

2.2 Configure Your Identity (One-time per machine)

```
# Your name (visible in commits)
git config --global user.name "Your Name"

# Your email (use the same as your GitHub account)
git config --global user.email "your.email@example.com"
```

Check your config:

```
git config --list
```

Other useful config:

```
# Set default branch name for new repos
git config --global init.defaultBranch main

# Cache credentials for 1 hour (save token)
git config --global credential.helper 'cache --timeout=3600'
```

2.3 Create a New Repository (From Scratch)

```
cd /path/to/your/project
git init
```

What it does: Turns the current folder into a Git repository. Creates a hidden `.git` folder.

2.4 Connect to GitHub (Add Remote)

```
git remote add origin https://github.com/Username/RepoName.git
```

What it does: Links your local repo to the GitHub repo. `origin` is the default name for the remote.

Check remotes:

```
git remote -v
```

2.5 .gitignore (Ignore Files You Don't Want in Git)

Create a `.gitignore` file in your project root to exclude files from being tracked:

```
# Create .gitignore
nano .gitignore
```

Common .gitignore patterns:

```
# OS files
.DS_Store
Thumbs.db

# IDE
.vscode/
.idea/

# Environment & secrets
.env
.env.local
*.env

# Dependencies
node_modules/
__pycache__/
*.pyc
venv/
.venv/

# Build output
dist/
build/
*.log
```

What it does: Git will ignore these files. They won't be added, committed, or pushed. Essential for keeping secrets (`.env`) and large folders (`node_modules`) out of the repo.

---

3. Basic Workflow — Daily Commands

This is what you'll use most often.

3.1 Check Status

```
git status
```

What it does: Shows which files are modified, staged, or untracked.

3.2 Add Files to Staging Area

```
# Add a specific file
git add filename.txt

# Add all changed files
git add .

# Add all files in a folder
git add folder/

# Interactive add (pick which changes to stage)
git add -p
```

What it does: Stages files so they're ready to be committed.

3.3 Commit (Save a Snapshot)

```
git commit -m "Your commit message here"
```

What it does: Saves a snapshot of all staged files with a message.

Good commit message examples: `- Add login feature - Fix bug in expense calculation - Update README with setup instructions`

3.4 Push (Upload to GitHub)

```
git push -u origin main
```

What it does: Sends your commits to GitHub. `-u origin main` sets up tracking so next time you can just use `git push`.

After first time:

```
git push
```

3.5 Pull (Download from GitHub)

```
git pull origin main
```

What it does: Downloads the latest commits from GitHub and merges them into your local branch.

Short form (if already tracking):

```
git pull
```

3.6 Basic Flow Summary

```
1. Make changes to files
2. git status          → See what changed
3. git add .           → Stage changes
```

```
4. git commit -m "msg" → Save snapshot
5. git push            → Upload to GitHub
```

---

4. Cloning & Remote Repositories

4.1 Clone (Get a Copy of a Repo)

When a team member wants to get the project:

```
git clone https://github.com/Username/RepoName.git
cd RepoName
```

What it does: Downloads the entire repo (including history) into a new folder.

Clone into a specific folder:

```
git clone https://github.com/Username/RepoName.git my-folder-name
```

4.2 Remote Commands

```
# List remotes
git remote -v

# Add a remote
git remote add origin https://github.com/User/Repo.git

# Change remote URL
git remote set-url origin https://github.com/User/Repo.git

# Remove a remote
git remote remove origin
```

4.3 Fetch vs Pull

Command	What it does
`git fetch origin`	Downloads new commits from GitHub. Does NOT merge. Safe.
`git pull origin main`	Fetch + merge in one step. Same as `git fetch` then `git merge`.

When to use fetch: When you want to see what's new before merging.

```
git fetch origin
git log HEAD..origin/main   # See new commits
git merge origin/main       # Merge when ready
```

Pull with rebase (cleaner history):

```
git pull --rebase origin main
```

---

5. Branching

Branches let you work on features without affecting the main code.

5.1 View Branches

```
# Local branches
git branch

# All branches (including remote)
git branch -a
```

5.2 Create a Branch

```
# Create and switch to new branch
git checkout -b feature-name

# Or (newer syntax)
git switch -c feature-name
```

Example:

```
git checkout -b add-voice-feature
```

5.3 Switch Between Branches

```
git checkout main
# or
git switch main
```

5.4 Push a Branch to GitHub

```
git push -u origin feature-name
```

What it does: Uploads your branch so team members can see it and open a Pull Request.

5.5 Delete a Branch

```
# Delete local branch
git branch -d feature-name

# Force delete (if not merged)
git branch -D feature-name
```

```
# Delete remote branch
git push origin --delete feature-name
```

---

6. Collaboration — Working with Your Team

6.1 Typical Workflow for Two People

User A: 1. Make changes → `git add .` → `git commit -m "message"` → `git push`

User B: 1. Before starting work: `git pull` (get User A's latest changes) 2. Make their changes 3. `git add .` → `git commit -m "message"` → `git push`

6.2 Sync Before You Start

Always pull before starting work:

```
git pull origin main
```

6.3 Push Your Changes

```
git add .
git commit -m "Add feature X"
git push origin main
```

6.4 If Someone Else Pushed First (Merge Conflict Possible)

```
git pull origin main
```

If there are **no conflicts**, Git merges automatically.

If there are **conflicts**, Git will tell you. See Section 12 for resolving them.

---

7. Pull Requests (PR)

A Pull Request is a way to propose changes before merging into the main branch.

7.1 When to Use PR

- Working on a **feature branch**
- Want **review** before merging
- Working in a **team**

7.2 PR Workflow

Step 1 — Create a feature branch:

```
git checkout -b add-ocr-feature
```

Step 2 — Make changes, commit, push:

```
git add .
git commit -m "Add OCR scanning feature"
git push -u origin add-ocr-feature
```

Step 3 — Open PR on GitHub: - Go to the repo on GitHub - Click **Compare & pull request** (or **Pull requests** → **New pull request**) - Select: `base: main` ← `compare: add-ocr-feature` - Add title and description - Click **Create pull request**

Step 4 — Review & Merge: - Another team member reviews the code - Click **Merge pull request** on GitHub - Optionally delete the branch after merge

Step 5 — Update your local repo:

```
git checkout main
git pull origin main
git branch -d add-ocr-feature
```

---

8. Merging & Rebasing

8.1 Merge (Combine Branches)

Merge a branch into your current branch:

```
git checkout main
git merge feature-branch
```

What it does: Combines the branch into `main`. Creates a merge commit.

8.2 Merge Types

Fast-forward merge: When there are no new commits on `main` since the branch was created. Git just moves the pointer.

Three-way merge: When both branches have new commits. Git creates a merge commit.

Squash merge (on GitHub): When merging a PR, choose "Squash and merge" to combine all commits into one. Keeps main branch history clean.

8.3 Rebase (Rewrite History — Advanced)

```
git checkout feature-branch
git rebase main
```

What it does: Replays your branch's commits on top of `main`. Makes history linear.

Warning: Don't rebase commits that have been pushed and shared. Use merge instead for shared branches.

---

9. Stash, Reset & Revert

9.1 Stash (Save Work Temporarily)

When you need to switch branches but have uncommitted changes:

```
# Save changes
git stash

# Or with a message
git stash save "WIP: half-done feature"
```

Restore later:

```
# Apply most recent stash
git stash pop

# Apply but keep in stash list
git stash apply

# List stashes
git stash list

# Apply specific stash
git stash apply stash@{0}
```

9.2 Reset (Undo Commits)

Soft reset — Keep changes, undo commit:

```
git reset --soft HEAD~1
```

Mixed reset — Unstage, keep file changes (default):

```
git reset HEAD~1
# or
git reset --mixed HEAD~1
```

Hard reset — Discard everything (destructive):

```
git reset --hard HEAD~1
```

`HEAD~1` = go back 1 commit. Use `HEAD~2` for 2 commits, etc.

9.3 Revert (Undo by Creating New Commit)

Safer for shared branches:

```
git revert HEAD
```

What it does: Creates a new commit that undoes the last commit. History stays intact.

---

10. Production Workflow

10.1 Branch Strategy (Recommended)

```
main        → Production-ready code
develop     → Integration branch (optional)
feature/*   → New features (e.g., feature/ocr-scan)
fix/*       → Bug fixes (e.g., fix/login-error)
```

10.2 Production Checklist

```
# Before merging to main
git checkout main
git pull origin main
git merge feature-branch
# Run tests here
git push origin main
```

10.3 Useful Production Commands

```
# View commit history
git log --oneline

# View who changed what
git blame filename

# Create a tag (for releases)
git tag -a v1.0.0 -m "Release version 1.0.0"
git push origin v1.0.0

# Show diff before committing
git diff
```

```
# Show diff of staged files
git diff --staged
```

---

11. Additional Essential Commands

Commands you'll need often — don't skip these.

11.1 Amend Last Commit (Fix Message or Add Forgotten File)

```
# Forgot to add a file? Add it and amend:
git add forgotten-file.txt
git commit --amend --no-edit

# Fix last commit message:
git commit --amend -m "New correct message"
```

What it does: Replaces your last commit instead of creating a new one. Use only if you haven't pushed yet, or on a private branch.

11.2 Reflog (Recover "Lost" Commits)

```
git reflog
```

What it does: Shows every move of HEAD. If you accidentally `reset --hard` and lose commits, use `git reflog` to find the old commit hash, then:

```
git reset --hard abc1234   # Use hash from reflog
```

11.3 Show a Specific Commit

```
git show abc1234
git show HEAD
```

What it does: Shows the commit message and file changes for that commit.

11.4 More git log Options

```
git log --oneline -10          # Last 10 commits
git log --oneline --graph      # Visual graph
git log --since="2 weeks ago"  # Recent commits
git log --author="Name"        # By author
git log -p filename            # History of one file with diffs
```

11.5 Compare Two Branches

```
# See diff between main and feature branch
git diff main..feature-branch
```

```
# See commits in feature branch not in main
git log main..feature-branch
```

11.6 Checkout a Remote Branch (Team Member Pushed New Branch)

```
# Fetch first
git fetch origin

# Create local branch tracking remote
git checkout -b feature-name origin/feature-name

# Or (newer)
git switch -c feature-name origin/feature-name
```

11.7 Cherry-Pick (Apply One Commit to Another Branch)

```
git checkout main
git cherry-pick abc1234
```

What it does: Takes one specific commit from another branch and applies it to your current branch.

11.8 Revert a Specific Commit

```
git revert abc1234
```

What it does: Creates a new commit that undoes the specified commit. Safer than reset for shared branches.

11.9 Remove Untracked Files (Clean)

```
# Preview what would be removed
git clean -n
# or
git clean --dry-run

# Remove untracked files
git clean -f

# Remove untracked files AND directories
git clean -fd
```

Warning: `git clean` permanently deletes untracked files. Use `-n` first to preview.

11.10 Rename a Branch

```
# Rename current branch
git branch -m new-name

# Rename other branch
git branch -m old-name new-name
```

11.11 Force Push (Use With Caution)

```
git push --force origin main
# or shorter
git push -f origin main
```

Warning: Overwrites remote history. Never use on shared branches unless you know what you're doing. Use `--force-with-lease` instead when possible (safer):

```
git push --force-with-lease origin main
```

11.12 More Stash Commands

```
git stash list              # List all stashes
git stash drop stash@{0}    # Delete a stash
git stash clear             # Delete all stashes
git stash show stash@{0}    # See stash contents
```

---

12. Common Scenarios & Solutions

Scenario 1: "I committed to wrong branch"

```
# Don't push yet! Move last commit to correct branch:
git reset --soft HEAD~1      # Undo commit, keep changes
git stash                    # Save changes
git checkout correct-branch  # Switch branch
git stash pop                # Apply changes
git add .
git commit -m "Your message"
```

Scenario 2: Merge conflict

```
git pull origin main
# Git says: CONFLICT in file.txt
```

1. Open the conflicted file. Look for:

   ```
   <<<<<<< HEAD
   your changes
   ```

```
=======
other team member's changes
>>>>>>> branch-name
```

2. Edit the file to keep what you want, remove the markers
3. Save the file
4. Run:

```
git add .
git commit -m "Resolve merge conflict"
git push
```

Scenario 3: "I need to undo my last push"

```
git revert HEAD
git push origin main
```

Scenario 4: Discard all local changes

```
# Unstage a file (keep changes)
git restore --staged filename
# or older syntax: git reset HEAD filename

# Discard changes in one file ( loses changes)
git restore filename
# or older syntax: git checkout -- filename

# Discard all local changes ( destructive)
git reset --hard HEAD
```

Scenario 5: See what changed in a file

```
git diff filename
git log -p filename
```

Scenario 6: "I lost my commits after reset –hard"

```
git reflog
# Find the commit hash (e.g., abc1234)
git reset --hard abc1234
```

Scenario 7: "I forgot to add a file to the last commit"

```
git add forgotten-file.txt
git commit --amend --no-edit
git push --force-with-lease   # Only if you already pushed!
```

Scenario 8: "A team member pushed a new branch, I need to get it"

```
git fetch origin
git checkout -b feature-name origin/feature-name
```

Scenario 9: "I need to undo multiple commits but keep changes"

```
git reset --soft HEAD~3   # Go back 3 commits, keep changes
```

---

13. Authentication: HTTPS vs SSH

HTTPS (Default)

- **URL:** `https://github.com/User/Repo.git`
- **Auth:** Username + Personal Access Token (PAT)
- **Token scope:** Enable `repo` when creating the token
- **Where to create token:** GitHub → Settings → Developer settings → Personal access tokens

SSH (No Password Popup After Setup)

Setup SSH key (one-time):

```
ssh-keygen -t ed25519 -C "your_email@example.com" -f ~/.ssh/id_ed25519 -N ""
```

Copy public key:

```
cat ~/.ssh/id_ed25519.pub
```

Add to GitHub: Settings → SSH and GPG keys → New SSH key → Paste key

Switch remote to SSH:

```
git remote set-url origin git@github.com:Username/RepoName.git
```

Cache Credentials (Save Token for 1 Hour)

```
git config --global credential.helper 'cache --timeout=3600'
```

---

14. Quick Reference Cheat Sheet

Setup & Clone

Command	Use
`git init`	Start new repo
`git clone <url>`	Copy repo from GitHub

Command	Use
`git remote add origin <url>`	Link to GitHub
`git remote -v`	List remotes
`.gitignore`	Ignore files (create file in root)

Daily Workflow

Command	Use
`git status`	See what changed
`git add .`	Stage all changes
`git add file`	Stage one file
`git add -p`	Interactive stage
`git commit -m "msg"`	Save snapshot
`git commit --amend`	Fix last commit
`git push`	Upload to GitHub
`git pull`	Download from GitHub
`git fetch origin`	Download without merge

Branches

Command	Use
`git branch`	List branches
`git branch -a`	List all (including remote)
`git checkout -b name`	Create & switch branch
`git checkout name`	Switch branch
`git branch -m new-name`	Rename branch
`git merge branch`	Merge branch into current
`git push -u origin branch`	Push new branch

Undo & Fix

Command	Use
`git stash`	Save work temporarily
`git stash pop`	Restore stashed work
`git stash list`	List stashes
`git restore --staged file`	Unstage file (keep changes)
`git restore file`	Discard changes in file
`git reset --soft HEAD~1`	Undo last commit (keep changes)
`git reset --hard HEAD~1`	Undo last commit (loses changes)
`git revert HEAD`	Undo last commit (new commit)

Command	Use
`git revert <hash>`	Undo specific commit

Info & History

Command	Use
`git log --oneline`	View commit history
`git log --oneline -10`	Last 10 commits
`git log --graph`	Visual graph
`git diff`	See unstaged changes
`git diff --staged`	See staged changes
`git show <hash>`	View specific commit
`git reflog`	Recover lost commits
`git blame file`	Who changed each line

Advanced

Command	Use
`git cherry-pick <hash>`	Apply one commit
`git clean -fd`	Remove untracked files
`git pull --rebase`	Pull with rebase
`git push --force-with-lease`	Force push (safer)

Happy coding! Share this guide with your team or university group.